



Laboratorio 04

Clasificadores de ensamble: Bagging, AdaBoost, Stacking y Gradient Boosting

Dr. Juan Bekios Calfa

Machine Learning

Universidad Católica del Norte

Primer semestre 2026

- Reutilizar la estrategia experimental del Lab03 sobre los mismos seis objetivos.
- Implementar clasificadores de ensamble con scikit-learn y un stacking robusto para clases pequeñas.
- Comparar Bagging, AdaBoost, Stacking y Gradient Boosting con validación cruzada anidada.
- Configurar hiperparámetros fuera del código mediante `config/paths.yaml`.
- Ejecutar primero `GridSearchCV` para todos los modelos y luego `RandomizedSearchCV`.
- Usar distribuciones reales para la búsqueda aleatoria.
- Generar tablas, matrices de confusión, métricas por clase, advertencias y archivos auditables.
- Conectar la práctica con la Clase 13: métodos de ensamble.

Idea clave

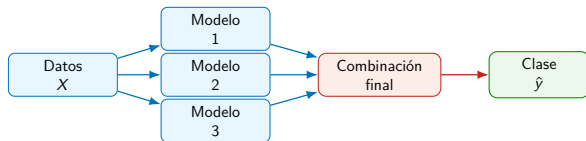
El foco no es entrenar “el mejor modelo”, sino construir una comparación defendible y reproducible entre estrategias de ensamble.

Idea de ensamble



Un ensamble combina varios modelos individuales para producir una decisión final.

- Bagging reduce varianza.
- AdaBoost corrige errores de forma secuencial.
- Stacking aprende cómo combinar predicciones.
- Gradient Boosting ajusta árboles sucesivos a los errores.



La predicción final actúa como un meta-clasificador.

Problema experimental



Entrada

Dataset SPSS con 15 atributos:

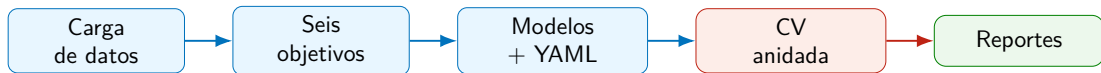
$$X \in \mathbb{R}^{n \times 15}$$

Los objetivos son: GDS, GDS_R1, GDS_R2, GDS_R3, GDS_R4 y GDS_R5.

Cada objetivo representa una reagrupación distinta de clases.



Objetivo	n_{min}	k_{outer}
GDS	2	2
GDS_R1	22	5
GDS_R2	172	5
GDS_R3	172	5
GDS_R4	64	5
GDS_R5	149	5



Se conserva

- Mismo dataset.
- Mismas salidas.
- Misma métrica de selección.
- Misma lógica de advertencias.
- Misma trazabilidad de resultados.

Se reemplaza

- Regresión logística base.
- Clasificadores pendientes.
- Núcleo del laboratorio por ensambles.
- Hiperparámetros movidos a YAML.

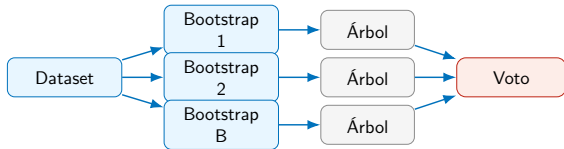
Bagging entrena varios clasificadores sobre muestras bootstrap:

$$D_b \sim \text{Bootstrap}(D), \quad b = 1, \dots, B$$

La predicción se obtiene por votación o combinación de probabilidades.

En el laboratorio

- `BaggingClassifier`.
- Árboles de decisión como modelos base.
- Grilla sobre estimadores, muestras y profundidad.



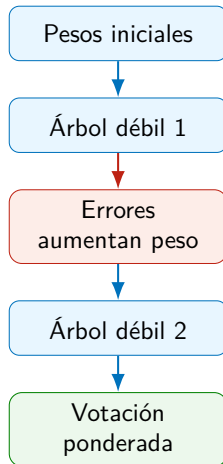
AdaBoost entrena clasificadores débiles de forma secuencial.

$$\epsilon_j = \sum_i w_i \mathbb{I}(y_i \neq \hat{y}_i)$$

$$\alpha_j = \frac{1}{2} \log \left(\frac{1 - \epsilon_j}{\epsilon_j} \right)$$

$$w_i \leftarrow w_i \exp(-\alpha_j y_i h_j(x_i))$$

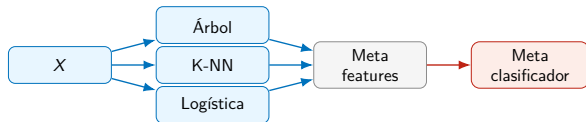
Los ejemplos difíciles pesan más en la ronda siguiente.



Stacking aprende un segundo nivel:

$$Z = [p_1(y|x), p_2(y|x), \dots, p_m(y|x)]$$

$$\hat{y} = g(Z)$$



En el laboratorio

- Base 1: árbol de decisión.
- Base 2: K-NN escalado.
- Base 3: regresión logística escalada.
- Meta-modelo: regresión logística.

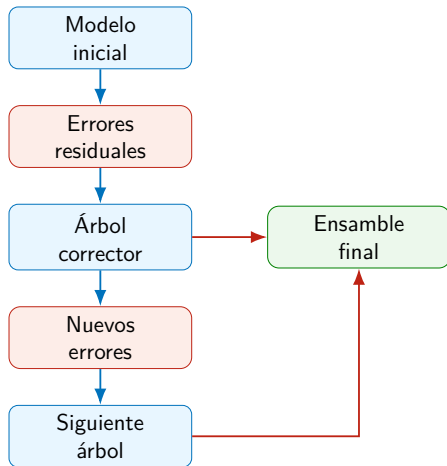
Gradient Boosting construye un ensemble secuencial:

$$F_m(x) = F_{m-1}(x) + \eta h_m(x)$$

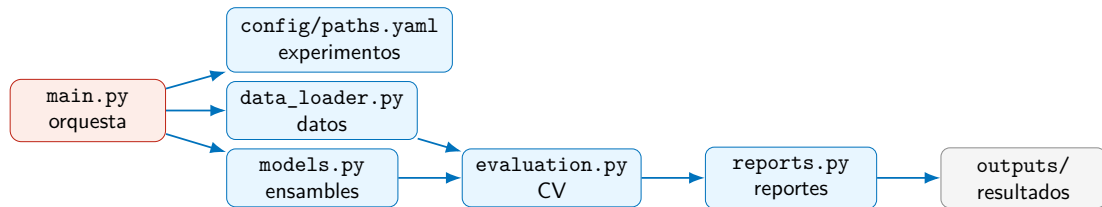
Cada árbol nuevo intenta corregir los errores del conjunto anterior.

En el laboratorio

- GradientBoostingClassifier.
- Búsqueda sobre número de árboles, tasa de aprendizaje, profundidad y subsample.
- Misma validación anidada que los demás modelos.



Estructura del repositorio



`models.py` define los pipelines; `config/paths.yaml` decide qué modelos se ejecutan y cómo se buscan sus hiperparámetros.

Experimentos desde YAML



```
experiment:
  experiments:
    - name: "grid_all"
      search_type: "grid"
      models:
        - "bagging"
        - "adaboost"
        - "stacking"
        - "gradient_boosting"

    - name: "random_all"
      search_type: "random"
      models:
        - "bagging"
        - "adaboost"
        - "stacking"
        - "gradient_boosting"
```

El orden del YAML define el orden didáctico: primero todas las grillas, luego todas las búsquedas aleatorias.

GridSearchCV

```
hyperparameter_search:  
  grid:  
    models:  
      bagging:  
        params:  
          clf__n_estimators: [25, 50]  
          clf__max_samples: [0.7, 1.0]
```

RandomizedSearchCV

```
hyperparameter_search:  
  random:  
    models:  
      bagging:  
        distributions:  
          clf__n_estimators:  
            dist: "randint"  
            low: 25  
            high: 151
```

Grid usa listas exhaustivas; Random usa distribuciones de `scipy.stats`.

GridSearchCV vs RandomizedSearchCV

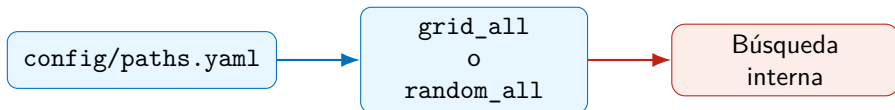


GridSearchCV

- Prueba todas las combinaciones.
- Es exhaustivo.
- Puede ser costoso si la grilla crece.
- Útil para grillas pequeñas y controladas.

RandomizedSearchCV

- Prueba solo `n_iter` combinaciones.
- Muestrea desde distribuciones.
- Usa una semilla para ser reproducible.
- Explora espacios más grandes con menor costo.
- En este laboratorio se usa en todos los ensambles dentro de `random_all`.

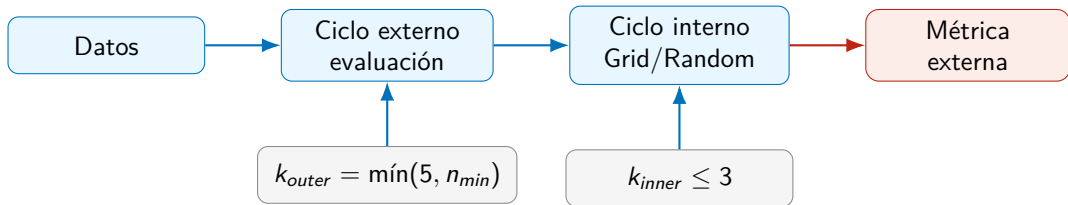


```
experiment:  
  random_state:  
    global: 42  
    outer_cv: 42  
    inner_cv: 123  
  model: 42  
  search: 42
```

- `outer_cv`: particiones externas.
- `inner_cv`: particiones internas.
- `model`: aleatoriedad de los modelos.
- `search`: búsqueda aleatoria.
- `global`: respaldo.

Idea clave

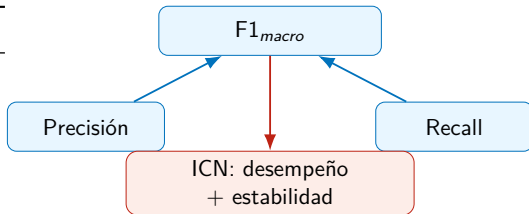
Centralizar las semillas evita que un modelo quede comparado bajo condiciones distintas por accidente.



Motivo

Separar selección de hiperparámetros y evaluación reduce el optimismo de una comparación hecha con una sola partición. El ciclo interno usa `f1_macro` y la búsqueda indicada en el YAML.

Métrica		Uso
F1 macro		Criterio principal ante desbalance.
Balanced accuracy		Recall promedio por clase.
Precisión macro		Pureza promedio.
Recall macro		Cobertura promedio.
ICN		Índice comparativo normalizado.



Con clases desbalanceadas, accuracy queda como métrica auxiliar.

Crear ambiente

```
conda env create -f environment.yml  
conda activate lab04_ml_2026_01
```

Ejecutar todo

```
python main.py
```

Ejecutar objetivos específicos

```
python main.py --targets GDS_R3 GDS_R5
```

Ejecutar modelos específicos

```
python main.py --models adaboost gradient_boosting
```

Ejecutar un experimento

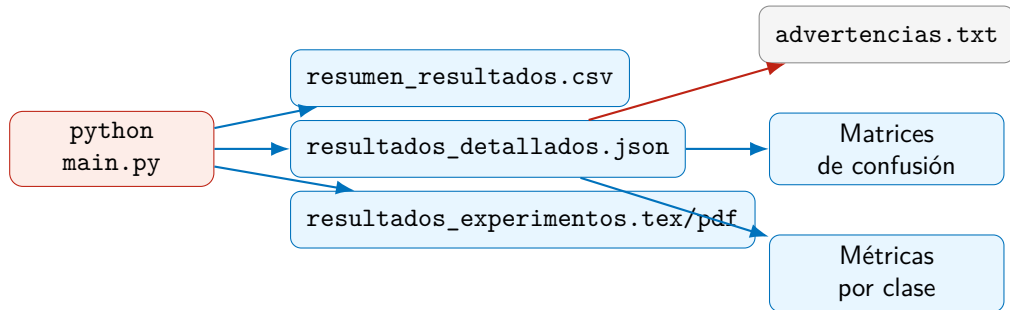
```
python main.py --experiments grid_all  
python main.py --experiments random_all
```

Revisar salidas

`outputs/tables/resumen_resultados.csv`
`outputs/advertencias.txt`

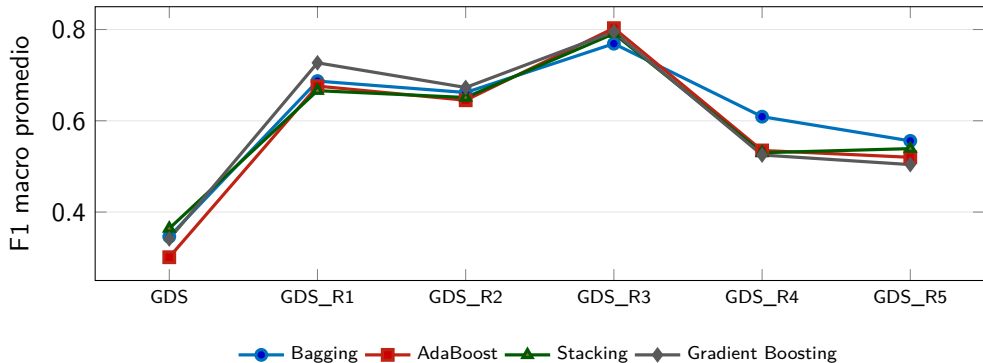
Columnas nuevas

`experiment_name`
`search_type`
`search_scoring`
`search_n_iter`
`best_params_mode`



El reporte principal se lee desde el CSV; los archivos por clase ayudan a explicar errores específicos.

Resultados: F1 macro



Lectura

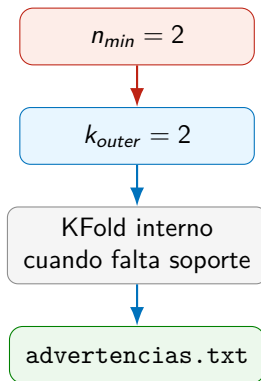
Se grafica, para cada modelo, su mejor F1 macro entre `grid_all` y `random_all`. No hay un ganador universal: el resultado depende del objetivo y de la estrategia de búsqueda.

Objetivo	Mejor F1 macro	Búsqueda	F1	Mejor ICN
GDS	Stacking	grid	0.364	Stacking / grid
GDS_R1	Gradient Boosting	random	0.727	Stacking / grid
GDS_R2	Gradient Boosting	grid	0.673	Bagging / random
GDS_R3	AdaBoost	random	0.803	Stacking / grid
GDS_R4	Bagging	random	0.609	Bagging / random
GDS_R5	Bagging	random	0.556	Stacking / grid

Interpretación

F1 macro premia el rendimiento promedio por clase; ICN incorpora además estabilidad entre folds y otras métricas macro. El ICN compara corridas completas: modelo más estrategia de búsqueda.

- En GDS, la clase menos frecuente tiene solo 2 ejemplos.
- El ciclo externo queda limitado a $k = 2$.
- En el entrenamiento de un fold puede quedar una clase con un solo ejemplo.
- El ciclo interno usa KFold no estratificado y registra advertencia.



Esta advertencia no invalida el laboratorio; indica que GDS debe interpretarse con cautela.

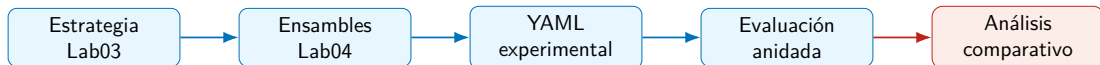
Checklist para el informe



- Describir dataset, atributos y seis objetivos.
- Explicar la diferencia entre Bagging, AdaBoost, Stacking y Gradient Boosting.
- Indicar qué modelos usan GridSearchCV y cuáles usan RandomizedSearchCV.
- Revisar las semillas `random_state` para asegurar reproducibilidad.
- Justificar validación cruzada anidada y uso de `f1_macro`.
- Reportar F1 macro, balanced accuracy, precisión, recall, estabilidad e ICN.
- Comparar ganadores por objetivo y discutir por qué cambian.
- Analizar al menos dos matrices de confusión.
- Incluir advertencias de soporte bajo cuando aparezcan.

Idea clave

Un buen informe explica comportamiento y límites, no solo ordena modelos por una métrica.



Mensaje final

Los ensambles no eliminan la necesidad de una evaluación cuidadosa; por eso la configuración, las semillas y la métrica deben quedar explícitas.